

字符串

第9章

主要内容

- 字符串的方法
- 字符串示例
- 字符串编码和解码(可选内容) : str <--> bytes

recap 字符串相关内置函数

转换为字符串

转换为数字类型

函数	含义	实例	结果
bin(x)	把数字x转换为二进制字符串	bin(4)	'0b100'
oct(x)	把数字x转换为八进制字符串	oct(25)	'0o31'
hex(x)	把数字x转换为十六进制字符串	hex(75)	'0x4b'
str(obj)	把对象obj转换为字符串	str(16), str('程序')	('16', '程序')
repr(obj)	把对象obj转换为内部表示的字符串	repr(16), repr('程序')	('16', '"程序"')
ascii(obj)	把对象obj转换为内部表示的ascii字符串	ascii('程序')	'"\u7a0b\u5e8f"'
int(x[,d])	把数字x截取为整数，或将基数为d的数字型字符串x转换为整数，如果d没有缺省为十进制	int(3.14), int('314') int('ff',16)	(3,314) 255
float(x)	将对象(数字或字符串)x转换为浮点数	float(3),float('3.14') float('12e-2')	(3.0,3.14) 0.12
len(obj)	返回序列类对象obj包含的元素个数	len('Hello World'),len('')	(11, 0)
ord(s)	返回长度为1的字符串s中的字符对应的Unicode码（ASCII码是其子集）	ord('a'),ord('A'),ord('你') ord('0'), ord('\n')	(97, 65, 20320) (48, 10)
chr(x)	返回Unicode编码为x的字符串(len=1)	chr(65),chr(10),chr(20320) chr(ord('D')-ord('A')+ord('a'))	('A', '\n', '你') 'd'

recap 字符串作为有序序列和iterable对象支持的方法

字符串是不可变的有序对象类型

- 作为有序序列对象：
 - 下标和切片访问
 - 连接运算 + 、重复运算 *
 - 比较运算
 - 求字符串长度 len
 - 有序对象支持的通用方法：判断**连续多个字符(子串)**出现在原有字符串中
 - 成员关系：in 和not in
 - 元素出现情况：index和count方法
- 作为iterable对象，当然也支持max/min/sorted/reversed/enumerate/zip等内置函数

描述	方法
去首尾字符 去前缀后缀	<code>strip([chars]) lstrip([chars])rstrip([chars])</code> <code>removeprefix(prefix)</code> <code>removesuffix(suffix)</code>
字符串类型判断	<code>isalnum()</code> <code>isalpha()</code> <code>isdecimal()</code> <code>isascii()</code> <code>isdigit()</code> <code>isidentifier()</code> <code>islower()</code> <code>isupper()</code> <code>isnumeric()</code> <code>isprintable()</code> <code>isspace()</code> <code>istitle()</code>
大小写转换	<code>lower()</code> <code>upper()</code> <code>swapcase()</code> <code>capitalize()</code> <code>title()</code> <code>casefold()</code>
测试和查找	<code>startswith(prefix[,start[,end]])</code> <code>endswith(suffix[,start[,end]])</code> <code>index(sub[,start[,end]])</code> <code>rindex(sub[,start[,end]])</code> <code>find(sub[,start[,end]])</code> <code>rfind(sub[,start[,end]])</code> <code>count(sub[,start[,end]])</code>
替换	<code>replace(old,new[,count])</code>
拆分和组合	<code>split(sep=None,maxsplit=-1)</code> <code>rsplit(sep=None, maxsplit=-1)</code> <code>partition(sep)</code> <code>rpartition(sep)</code> <code>splitlines([keepends])</code> <code>join(iterable)</code> (已介绍)
填充和对齐 (已介绍)	<code>zfill(width)</code> <code>center(width[,fillchar])</code> <code>ljust(width[,fillchar])</code> <code>rjust(width[,fillchar])</code> <code>expandtabs([tabsize])</code>
翻译和转换	<code>maketrans</code> <code>translate</code>
格式化 (已介绍)	<code>format(*args, **kwargs)</code>

recap: 字符串填充和对齐示例: 图案问题

05-循环结构: for循环: 图案问题

```
total = int(input('请输入n:'))
for i in range(total):
    for j in range(i + 1):
        print('*', end='')
    print()
```

```
for i in range(total):
    stars = '*' * (i + 1)
    print(stars)
```

```
for i in range(0, total):
    stars = ' '.join('*' * (i + 1))
    print(stars)
```

```
# n stars + n-1 spaces
for i in range(total):
    stars = ' '.join('*' * (i + 1))
    line = stars.rjust(total * 2 - 1, ' ')
# line = '{: >{}}'.format(stars, lines * 2 - 1)
    print(line)
```

```
*
**
***
****
*****
```

```
      *
    * *
  * * *
* * * *
* * * * *
```

```
      *
    * *
  * * *
* * * *
* * * * *
```

字符串的元素访问和计数

```
>>> 'pppppp'.count('pp')
3
```

字符串是有序序列，可以通过下标访问指定位置的元素(即由对应位置的字符组成的字符串)，可支持in, count, index, 只是其参数除了是单个字符组成的字符串外，还可是长度超过1的**子字符串**

- `strobj.index(sub[,start[,end]])` 返回子串sub(而不仅仅是单个字符)在指定范围内**首次出现**的位置(下标)，**如果不存在则抛异常ValueError**
- `strobj.count(sub[,start[,end]])` 返回子串sub出现的次数，找到子串后，**在之后的下一个位置**继续...
- `sub in S` `sub not in S` 判断子串sub是否在S中出现
- `rindex(sub[,start[,end]])` 返回子串sub在指定范围内**最后一次**出现的位置，**如果不存在则ValueError**
- `find(sub[,start[,end]])` `rfind(sub[,start[,end]])` 与index/rindex类似，返回子串sub在指定范围内首次或最后一次出现的位置，**如果不存在则返回-1**

```
>>> s = "apple,peach,banana,peach,pear"
>>> s.index('peach')
6
>>> s.index('ppp')
Traceback (most recent call last):
...
ValueError: substring not found
>>> s.count('an')
2
>>> s.rindex('peach')
19
```

```
>>> s.find("peach")
6
>>> s.find("peach", 6 + 5) # 6 + len('peach')
19
>>> s.find("peach", 11, -1)
19
>>> s.rfind('peach')
19
>>> s.rfind('ppp')
-1
```

字符串的方法：去首尾字符

`strip([chars])` `lstrip([chars])` `rstrip([chars])`

- 将字符串的两端、左端或右端的指定字符(chars中的那些字符)去掉，返回一个新的字符串
- 找到第一个不在chars的字符，之前的字符移走
- 类似地，找到倒数第一个不在chars的字符，之后的字符移走
- 不传递参数，等价于chars为' \t\n\r\x0b\x0c'，即空格类字符

```
>>> s2 = 'www.example.com'
>>> s2.lstrip('wcmo.')
'example.com'
>>> s2.rstrip('wcmo.')
'www.example'
>>> s2.strip('wcmo.')
'example'
```

```
>>> s = '  python programming\n '
>>> s.strip()
'python programming'
>>> s.lstrip()
'python programming\n '
>>> s.rstrip()
'  python programming'
```

```
>>> '{1456};'.strip('{};')
'1456'
```

思考：自己实现strip()?

字符串的方法: startswith/endswith

`startswith(prefix[,start[,end]])` `endswith(suffix[,start[,end]])`

判断字符串或者指定范围的字符串是否以prefix开头或suffix结尾

- prefix和suffix可以是字符串
- **也可以是一个元组**, 其每个元素为一个字符串, 表示是否其中任意一个字符串开头或结尾

```
addrs = 'tom@fudan.edu.cn', 'tom@m.fudan.edu.cn'
for addr in addrs:
    if addr.endswith(('fudan.edu.cn', 'm.fudan.edu.cn')):
        print('Fudan Email address:', addr)
```

- `startswith`: 取strobj的前面最多`len(prefix)`字节 `strobj[:len(prefix)] == prefix`
- `endswith`: 取strobj的尾部最多`len(suffix)`字节 `strobj[-len(suffix):] == suffix`

字符串的方法：去前缀或后缀(了解)

`removeprefix(prefix)` `removesuffix(suffix)`

- Python3.9引入，以prefix开始时或者以suffix结束时，去除该前缀或者后缀，返回新的字符串。如果没有指定的前缀或者后缀，返回原来的字符串

```
>>> s = '/home/dlmao/test.py'
>>> s.removeprefix('/home/')
'dlmao/test.py'
>>> s.removesuffix('.py')
'/home/dlmao/test'
>>> s.removesuffix('.exe')
'/home/dlmao/test.py'
```

```
def removeprefix(text, prefix):
    if not prefix:
        return text
    if text.startswith(prefix):
        # text[:len(prefix)] == prefix
        return text[len(prefix) :]
    else:
        return text
```

- **思考：**自己实现removesuffix?

字符串的方法：类型判断，根据Unicode字符集中的类型判断

string模块中的相关属性给出了ASCII字符集中的字母、数字、空格等

isdecimal()	非空字符串的所有字符都为十进制数字(还包括全角数字及ARABIC-INDIC十进制数字等)
isdigit()	非空字符串的所有字符都为digit(还包括十进制数字的上标、下标、各种圆圈、括号、点形式等)
isnumeric()	非空字符串的所有字符都为numeric(还包括罗马数字, 中文数字, 分数等)
isalpha()	非空字符串的所有字符都为字母(Unicode的Letter, 包括汉字) 时返回True
isalnum()	非空字符串的字符都为字母或者numeric
isascii()	空字符串, 或者字符串的字符都为ASCII字符集中的字符
isidentifier()	非空字符串是否为合法的python标识符
islower() isupper()	非空字符串的那些可区分大小写 (比如汉字不区分大小写) 的字符是否都为小写或大写字母
isprintable()	空字符串, 或者字符串的字符都为可打印(可见) 字符 (\t\n等控制字符不是可打印字符)
isspace()	非空字符串的字符都为空格类字符 (空格、制表符\t、回车\n等)
istitle()	非空字符串中的每个单词 (单词类字符: 英文字母) 的首字母是否为大写

- 要求所有的字符都满足指定的特性，而不是仅仅其中某个字符满足
- is类方法一般要求至少有一个属于那一类型的字符，除了isascii和isprintable
- 数字字符的范围： decimal < digit < numeric

```
>>> ''.isspace()  
False
```

十进制数字

decimal < digit < numeric

下面给出了一些典型的decimal, digit, numeric字符的例子 (并没有全部列出)

numeric.py

```
DECIMALS = '0123456789 0 1 2 3 4 5 6 7 8 9 '  
DIGITS = ('0123456789 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 ①②③④⑤⑥⑦⑧⑨ '  
          '(1)(2)(3)(4)(5)(6)(7)(8)(9)1 2 3 4 5 6 7 8 9 ①②③④⑤⑥⑦⑧⑨ '  
          '①②③④⑤⑥⑦⑧⑨①②③④⑤⑥⑦⑧⑨')  
NUMERICS = ('0123456789 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 ①②③④⑤⑥⑦⑧⑨ '  
            '⑩⑪⑫⑬⑭⑮⑯⑰⑱⑲⑳(1)(2)(3)(4)(5)(6)(7)(8)(9)(10)(11)(12)(13)(14)(15)(16)(17)(18)(19)(20) '  
            '1 2 3 4 5 6 7 8 9 10.11.12.13.14.15.16.17.18.19.20.①②③④⑤⑥⑦⑧⑨⑩ '  
            '①②③④⑤⑥⑦⑧⑨⑩①②③④⑤⑥⑦⑧⑨⑩ '  
            'ⅠⅡⅢⅣⅤⅥⅦⅧⅨⅩⅪⅫⅬⅭⅮⅰⅱⅲⅳⅴⅵⅶⅷⅸⅹⅺⅻⅼⅽⅾⅿ i ii iii iv v vi vii viii ix x xi xii l c d m '  
            '1/7 1/9 1/10 1/3 2/3 1/5 2/5 3/5 4/5 1/6 5/6 1/8 3/8 5/8 7/8 '  
            '〇一二三四五六七八九十百千万亿 '  
            '零壹贰叁肆伍陆柒捌玖拾佰仟万亿')
```

```
>>> '1234'.isdecimal()
```

True

```
>>> '1234'.isdigit()
```

True

```
>>> '①'.isdecimal()
```

False

```
>>> '①'.isdigit()
```

True

```
>>> '一千三百'.isdigit()
```

False

```
>>> '一千三百'.isnumeric()
```

True

字符串的方法：类型判断，根据Unicode字符集中的类型判断

```
# 空字符串或...
>>> ''.isascii()
True
>>> ''.isprintable()
True

>>> '一千三百'.isascii()
False
>>> '1234abcDE'.isascii()
True

>>> 'ABC DE?'.isprintable()
True
>>> 'ABC\tDE?'.isprintable()
False

# 要求非空字符串且...
>>> ' '.isspace()
False
>>> '\t\n'.isspace()
True
```

```
>>> 'abcd'.isalpha()
True
>>> '1234'.isalpha()
False
>>> '一千三百'.isalpha()
True

>>> '1234abcDE'.isalnum()
True

>>> '1234abcDE'.isidentifier()
False
>>> '中国一千三百'.isidentifier()
True
```

```
>>> '1234abcDE'.islower()
False
>>> '1234 abcde'.islower()
True
>>> 'ABC DE?'.isupper()
True

# 连续的英文字母组成单词
>>> 'How Are You?'.istitle()
True
```

字符串的方法：大小写转换

<code>lower()</code>	返回小写格式的字符串
<code>upper()</code>	返回大写格式的字符串
<code>capitalize()</code>	返回字符串, 首字母大写 , 其他字母小写
<code>title()</code>	返回字符串, 每个单词(英文字母组成的单词) 的首字母大写, 其他小写
<code>swapcase()</code>	返回字符串, 大小写互换

自己实现大小写英文字母转换swapcase

swapcase.py

```
def swapcase(text):
    text2 = []
    for ch in text:
        if 'a' <= ch <= 'z':
            text2.append(chr(ord(ch) - ord('a') + ord('A')))
        elif 'A' <= ch <= 'Z':
            text2.append(chr(ord(ch) - ord('A') + ord('a')))
        else:
            text2.append(ch)
    return ''.join(text2)
```

```
>>> s = "What is YoUr NaMe?"
>>> s.lower()
'what is your name?'
>>> s.upper()
'WHAT IS YOUR NAME?'
>>> s.capitalize()
'What is your name?'
>>> s.title()
'What Is Your Name?'
>>> s.swapcase()
'wHAT IS yOuR nAmE?'
```

a	b	c	...	x	y	z
A	B	C		X	Y	Z
0	1	2	...	23	24	25

字符串的方法: replace 替换

replace(old,new[,count])

- 返回新的字符串，原始字符串中的所有子串old被替换为new。如果count给出，则替换前面的最多count个子串
- new为空字符串时，删除子串

```
>>> s = 'apple, peach, banana, peach, pear'
>>> s.replace('peach', 'PEACH')
'apple, PEACH, banana, PEACH, pear'
>>> s.replace('peach', 'PEACH', 1)
'apple, PEACH, banana, peach, pear'
>>> s.replace(',', ' ')
'apple peach banana peach pear'
>>> 'a b c'.replace(' ', '')
'abc'
```

```
>>> filename = 'C:/Program Files/Microsoft Office/Office16/winword.exe'
>>> filename.replace('/', '\\')
'C:\\Program Files\\Microsoft Office\\Office16\\winword.exe'
```

字符串的方法: maketrans/translate 并行转换

- replace方法只能串行替代, 而不能并行替代! `s.replace(',', ' ').replace(';', ' ')`
 - 比如 逗号和分号对调! 错误的实现: `s.replace(',', ';').replace(';', ',')`
- 将字符串中的某些单字符并行替代为另外一些字符 (或字符串) 或删除
 - 首先maketrans构造映射表, 然后translate进行实际的替换
- 静态方法 `str.maketrans(x, y[, z])` 如果有两个以上参数, x和y必须长度一致, 每一个在x出现的字符转换为y对应位置的字符, 如果有第三个参数, 表示在z中出现的字符转换为None, 即删除
- `strobj.translate(table)` 按照映射表对strobj进行转换, 返回转换后的新字符串

```
>>> s = 'Great hopes make great man.'
>>> tab = str.maketrans('abcde', 'uvwxy')
>>> ss = s.translate(tab)
>>> ss
'Gryut hopys muky gryut mun.'
>>> tab2 = str.maketrans('uvwxy', 'abcde')
>>> ss.translate(tab2)
'Great hopes make great man.'

>>> tab3 = str.maketrans('abcde', 'uvwxy', '.*?')
>>> s.translate(tab3)
'Gryut hopys muky gryut mun'
```

```
>>> s1 = '我的银行账号为3141516,密码为123456'
>>> tab4 = str.maketrans('0123456789', '*' * 10)
>>> s1.translate(tab4)
'我的银行账号为*****,密码为*****'
```

字符串的方法: maketrans/translate 并行转换

- maketrans创建的映射表, 实际上就是1个字典, 该字典的key为字符的unicode(整数), 而value为
 - 如果为整数, 则是要替换为的字符的Unicode
 - 如果为None, 则表示删除
 - 如果为字符串, 则原来的字符替换为相应字符串
- maketrans(x,y[,z]) x和y的长度相同, 每一个在x出现的字符转换为y对应位置的字符, 在z出现的字符被删除
- maketrans(mapping), 只传递一个字典参数, 将某个字符映射为一个字符或多个字符或None
 - 该字典的key应该为长度为1的字符串ch, 也可以是ord(ch)返回的整数(即unicode码点)
 - 值可以是:
 - None, 表示删除该字符
 - 字符串, 表示替代为该字符串
 - 整数n, 表示替换为chr(n)

```
>>> str.maketrans('abcde', 'uvwxy', '.*?')
{97: 117, 98: 118, 99: 119, 100: 120,
101: 121, 46: None, 63: None}
```

```
>>> s2 = 'dlmao@fudan.edu.cn'
>>> tab5 = str.maketrans({'@': ' AT ', '.': ' DOT '})
>>> tab5
{64: ' AT ', 46: ' DOT '}
>>> s2.translate(tab5)
'dlmao AT fudan DOT edu DOT cn'
```

字符串的方法：分割 split和rsplit

`split(sep=None, maxsplit=-1)` `rsplit(sep=None, maxsplit=-1)`

- 以指定字符串`sep`为分隔符分割字符串，返回一个字符串列表，其元素为分割出来的字符串（不包括分隔符）。如果字符串中`sep`连续出现，或者`sep`出现在开始或结束位置，会分割出空字符串
- `sep`一般为单个字符，但也可以是多个字符
- 如果`maxsplit`给出且大于-1，则最多只进行`maxsplit`次分割，即列表元素个数最多为`maxsplit+1`。如果小于等于-1，则不限次数
- `rsplit`函数表示从右边开始进行分割，但结果仍然按照出现在原字符串的顺序排列

```
>>> s = '1 2 3'
>>> s.split(' ')
['1', '2', '3']
>>> s.rsplit(' ')
['1', '2', '3']
```

```
>>> fruits = " apple, banana ,orange,grape,,, pear ,peach ,"
>>> fruits.split(',')
[' apple', ' banana ', 'orange', 'grape', '', '', ' pear ', 'peach ', '']
>>> [item.strip() for item in fruits.split(',') if item]
['apple', 'banana', 'orange', 'grape', 'pear', 'peach']
>>> fruits.split(',', 2)
[' apple', ' banana ', 'orange,grape,,, pear ,peach ,']
>>> fruits.split(' ', 1)
[' apple', ' banana ,orange,grape,,, ', 'pear ,peach ,']
```

```
>>> t = "2014-10-31"
>>> t.split('-')
['2014', '10', '31']
>>> [int(c) for c in t.split('-')]
[2014, 10, 31]
>>> t.rsplit('-')
['2014', '10', '31']
>>> t.rsplit('-', 1)
['2014-10', '31']
>>> ''.split(',')
['']
```

字符串的方法：分割 split和rsplit

`split(sep=None, maxsplit=-1)` `rsplit(sep=None, maxsplit=-1)`

- 以指定字符串`sep`为分隔符分割字符串，返回一个字符串列表，其元素为分割出来的字符串（不包括分隔符）。如果字符串中`sep`连续出现，或者`sep`出现在开始或结束位置，会分割出空字符串
- `sep`一般为单个字符，但也可以是多个字符
- **如果`sep==None`:**
 - 所有空白类字符（包括空格、换行符、制表符等等）都将被认为是分隔符
 - 多个连续的空白类字符当成一个整体，即相当于1个空白分隔符。中间不会分割出空字符串
 - 字符串最前面和最后面分割出来的空字符串会被移走
 - **所有空字符串会被移走，每个字符串首尾都没有空格类字符**

```
>>> s2 = '\nHello world \n\n My name is Mike'
>>> s2.split()
['Hello', 'world', 'My', 'name', 'is', 'Mike']
>>> s2.split(' ')
['\nHello', 'world', '\n\n', 'My', '', 'name', 'is', 'Mike', '']
>>> ''.split() # 目的是得到所有（非空格类字符组成的）单词
[]
```

```
>>> ''.split(' ')
['']
```

字符串的方法：分割 splitlines

splitlines([keepends=False])

- 根据换行类字符(包括'\r', '\n', '\r\n')将字符串分割成多个子字符串，返回这些**子字符串的列表**。
- 字符串最后为换行字符时，在该分隔符之后不会出现空字符串
- 空字符串调用本函数时返回空列表
- keepends=True表示保留分隔符(换行字符)

	splitlines()	split('\n')
空字符串	返回空列表	返回包含空字符串的列表
最后为\n	该分隔符\n后面没有空字符串	该分隔符后面会分割出一个空字符串
keepends	可保留换行字符	换行符不保留

```
>>> ''.split(' ')
['']
```

```
>>> ''.split() # (非空格类字符组成的) 单词
```

```
[]
```

```
>>> ''.splitlines()
```

```
[]
```

```
>>> s = '\nline 2\n\nline 4\n'
>>> print(s)
```

```
line 2
```

```
line 4
```

```
>>> s.split('\n')
['', 'line 2', '', 'line 4', '']
>>> ''.split('\n')
['']
```

```
>>> s.splitlines()
['', 'line 2', '', 'line 4']
>>> s.splitlines(True)
['\n', 'line 2\n', '\n', 'line 4\n']
>>> ''.splitlines()
[]
```

字符串的方法： partition rpartition 分割一次

partition(sep)

rpartition(sep)

- 以指定字符串sep为分隔符**分割最多一次**，将原字符串分割为**包括3部分的元组**，即sep前的字符串、sep、sep后的字符串
 - 如果指定的分隔符不在原字符串中，则返回原字符串 + 两个空字符串
- rpartition表示从右边开始分割，但是返回的顺序仍然是原字符串中的顺序
 - 如果指定的分隔符不在原字符串中，则返回两个空字符串 + 原字符串

```
>>> fruits = " apple, banana ,orange,grape,,, pear ,peach ,"
>>> fruits.partition(',')
(' apple', ',', ' banana ,orange,grape,,, pear ,peach ,')
>>> fruits.rpartition(',')
(' apple, banana ,orange,grape,,, pear ,peach ', ',', ' ')
>>> fruits.partition('\n')
(' apple, banana ,orange,grape,,, pear ,peach ', '\n', '')
>>> fruits.rpartition('\n')
('', '', ' apple, banana ,orange,grape,,, pear ,peach ,')
```

```
first, sep, second = s.partition('=')
if sep:
    print(first, second)
```

字符串的方法: join 组合多个字符串

strobj.join(iterable)

- 将iterable对象(列表,字符串等)中的多个元素组合在一起,元素间插入相应的**分割字符串strobj**, 返回新的字符串
- iterable对象中的**元素必须是字符串**

```
>>> s = list('abc')
>>> s
['a', 'b', 'c']
>>> ''.join(s)
'abc'
>>> ','.join(s)
'a,b,c'
>>> print('\n'.join(s))
a
b
c
>>> s2 = list(range(5))
>>> s2
[0, 1, 2, 3, 4]
>>> '+'.join(str(i) for i in list2)
'0+1+2+3+4'
```

字符串示例1

问题：统计多行字符串的行数

- 统计\n的出现次数：要考虑最后一行可能不是以换行符结束的情形
 - 算术运算中True相当于1， False相当于0
- splitlines分割成多行

count_line.py

```
def line_count(text):  
    return text.count("\n") + (text[-1] != "\n")  
    # return text.count("\n") + (1 if text[-1] != "\n" else 0)  
  
def line_count2(text):  
    return len(text.splitlines())  
  
if __name__ == "__main__":  
    text = """key1=value1  
              key2 = value2  
              key3 = value3= """  
    print(line_count(text))  
    print(line_count2(text))  
  
    print(line_count(text + "\n"))  
    print(line_count2(text + "\n"))
```

字符串示例2

- 配置文件解析：比如 key = value。可采用index, find, partition(split) 方法。

```
text = '''key1=value1
key2 = value2
key3 = value3=
'''
```

parse_config.py

```
def parse_config(text):
    for line in text.splitlines():
        if "=" in line: # 也可以采取异常处理try机制实现
            i = line.index("=")
            key, value = line[:i].strip(), line[i + 1 :].strip()
            print("key={} value={}".format(key, value))
```

```
def parse_config2(text):
    for line in text.splitlines():
        if (i := line.find('=')) != -1:
            key, value = line[:i].strip(), line[i + 1 :].strip()
            print("key={} value={}".format(key, value))
```

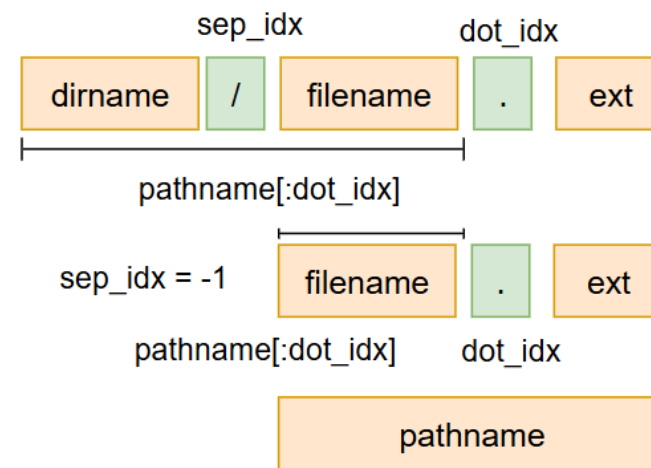
```
def parse_config3(text):
    for line in text.splitlines():
        key, sep, value = line.partition("=")
        if sep:
            print("key={} value={}".format(key.strip(), value.strip()))
```

```
if '=' in line:
    key, value = line.split('=', 1)
```

字符串示例3

路径名pathname分割为(root, ext)

- 扩展名是路径名的最后的文件名(filename)中最后一个点之后的内容
- 如果最后的文件名开始到最后一个点之间都是点, 返回pathname, "



如果filename的每个字符为dot, 返回(pathname, "
`pathname[sep_idx+1:dot_idx]`)

split_ext.py

```
def split_ext(pathname, sep="/", extsep="."):  
    sep_idx = pathname.rfind(sep)  
    dot_idx = pathname.rfind(extsep)  
    if dot_idx > sep_idx:  
        filename_idx = sep_idx + 1  
        while filename_idx < dot_idx:  
            if pathname[filename_idx] != extsep:  
                return pathname[:dot_idx], pathname[dot_idx:]  
            filename_idx += 1  
    return pathname, ""
```

```
if pathname[filename_idx:dot_idx] != extsep * (dot_idx - filename_idx):  
    return pathname[:dot_idx], pathname[dot_idx:]
```

字符串示例4 [方法1] maketrans+translate

问题9-1 使用凯撒密码对明文中的英文字母替代为偏移指定距离的英文字母。最后一个字符z的下一个位置为从头开始的a。偏移13位置也称为ROT13

a	b	c	...	x	y	z
A	B	C		X	Y	Z
0	1	2	...	23	24	25

- [方法1] maketrans+translate [方法2] 每个字符计算偏移位置

```
def caesar_encode_translate(text, offset=13):
```

```
    offset = offset % 26
```

```
    before = string.ascii_letters
```

```
    after = (string.ascii_lowercase[offset:] + string.ascii_lowercase[:offset] +  
            string.ascii_uppercase[offset:] + string.ascii_uppercase[:offset])
```

```
    table = str.maketrans(before, after)
```

```
    return text.translate(table)
```

```
def caesar_decode_translate(text, offset=13):
```

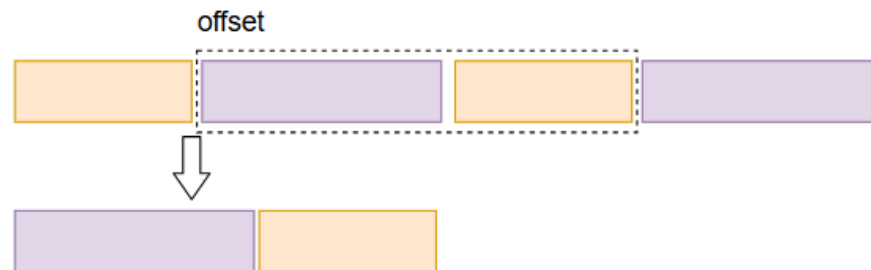
```
    offset = offset % 26
```

```
    before = string.ascii_letters
```

```
    after = (string.ascii_lowercase[offset:] + string.ascii_lowercase[:offset] +  
            string.ascii_uppercase[offset:] + string.ascii_uppercase[:offset])
```

```
    table = str.maketrans(after, before)
```

```
    return text.translate(table)
```



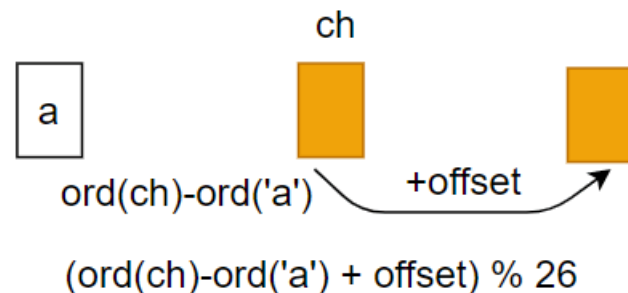
caesar.py

字符串示例4: [方法2] 每个字符逐个处理, 决定其偏移位置

- 问题9-1 使用凯撒密码对明文中的英文字母替代为偏移指定距离的英文字母。最后一个字符z的下一个位置为从头开始的a

```
def caesar_encode(text, offset=13):  
    offset = offset % 26  
    encrypted = []  
    for char in text:  
        if char in string.ascii_lowercase:  
            idx = (ord(char) - ord('a') + offset) % 26  
            encrypted.append(string.ascii_lowercase[idx]) # chr(ord('a') + idx)  
        elif char in string.ascii_uppercase:  
            idx = (ord(char) - ord('A') + offset) % 26  
            encrypted.append(string.ascii_uppercase[idx])  
        else:  
            encrypted.append(char)  
  
    return ''.join(encrypted)  
  
def caesar_decode(crypt, offset=13):  
    offset = offset % 26  
    return caesar_encode(crypt, -offset) # or 26 - offset
```

a	b	c	...	x	y	z
A	B	C		X	Y	Z
0	1	2	...	23	24	25



caesar.py

字符串示例5

问题9-2: 密码的安全强度, 根据是否包含数字、小写字母、大写字母和可打印标点符号评分[0,4]

points[i]

- 对应4种类别
- True or False: 是否包含指定类别的字符

```
def check_password_strength(pwd):  
    SCORE_TO_DESC = {  
        0: "Error",          1: "Weak",          2: "Bellow Middle",  
        3: "Above Middle", 4: "Strong",  
    }  
    if not isinstance(pwd, str):  
        raise ValueError("not a valid string")  
    if len(pwd) < 6:  
        raise ValueError("must contains 6 chars at least.")  
    points = [False] * 4  
    for ch in pwd:  
        if not points[0] and ch in string.digits:  
            points[0] = True  
        elif not points[1] and ch in string.ascii_lowercase:  
            points[1] = True  
        elif not points[2] and ch in string.ascii_uppercase:  
            points[2] = True  
        elif not points[3] and ch in ",.!?;<>": # string.punctuation  
            points[3] = True  
    score = sum(points)  
    return SCORE_TO_DESC[score]
```

password_strength.py

拓展：中文字符串格式化：中文字符占2个英文字符的宽度

```
import unicodedata
def east_str_width (text):
    """ 返回text等价的英文字符的宽度 """
    t = [ 2 if unicodedata.east_asian_width(ch) in
'WFA' else 1 for ch in text ]
    return sum(t)
```

format_eastasia.py

```
def format_east_str(text, min_width=10, justify=None,
fillchar=' '):
    width = east_str_width (text)
    if width >= min_width: return text
    fill_width = min_width - width
    if justify == '>' : # right justified
        return fill_width * fillchar + text
    elif justify is None or justify == '<':
        return text + fill_width * fillchar
    else: # center
        return (fill_width+1) // 2 * fillchar + text
+ fill_width // 2 * fillchar
```

```
format_east_str('中国12上海', 20, '^', '*')
```

*****中国12上海*****

```
# 中文unicode码 [0x4e00, 0x9fa5]
>>> 0x4e00 <= ord('中') <= 0x9fa5
True
>>> 0x4e00 <= ord('a') <= 0x9fa5
False
```

east_asia_width	意义
A: Ambiguous	不确定
F: Fullwidth	全宽
H: Halfwidth	半宽
N: Neutral	中性
Na: Narrow	窄
W: Wide	宽

format_east_str函数按照相应的格式格式化字符串**text**

- 最小宽度缺省为10,
- 对齐方式缺省为左对齐, 可取的值 '<', '>', '^', None
- 填充字符缺省为空格

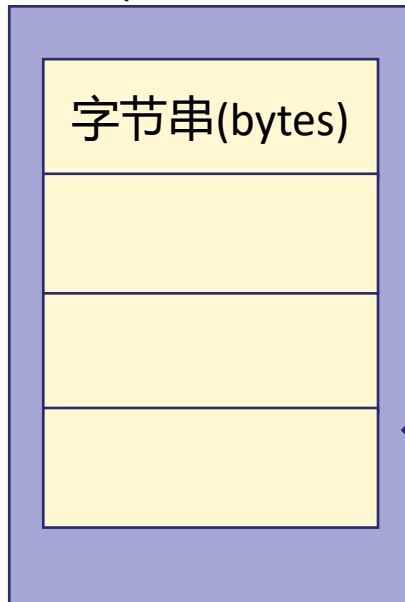
主要内容

- 字符串的方法
- 字符串示例
- **字符串编码和解码（可选内容）：str <--> bytes**

字符串编码和解码

- 保存在外部存储(硬盘/U盘等) 中的数据以字节方式保存
- 我们通过IDLE等编写的源文件(.py), 其编码方式缺省为UTF-8编码。
 - 编辑器在加载文件时, 基于UTF-8编码方式, 将外部存储的字节流解释为Unicode字符集中的字符串str
 - 编辑器在保存文件时, 基于UTF-8编码方式, 将字符串编码成字节流后保存到外部存储
- **类型bytes**类似于str, 也是一个**不可变**的有序序列类型, 只是**其元素为单个字节**, 而str中每个元素为单个字符
 - 字面量定义: 和str类似, 只是在之前添加**b或者B**即可, 比如b'abcd'
- 编码方式(encoding): UTF-8, ASCII, Latin1, GBK等
- errors: 在编码/解码过程中**无法转换时怎么办**
 - strict: 抛异常 ValueError(UnicodeEncodeError或UnicodeDecodeError)
 - ignore: 忽略
 - replace: 替代以特殊字符?
 - ...

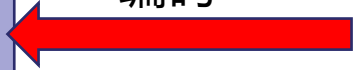
文件(UTF-8或其他编码)



读文件
解码

写文件
编码

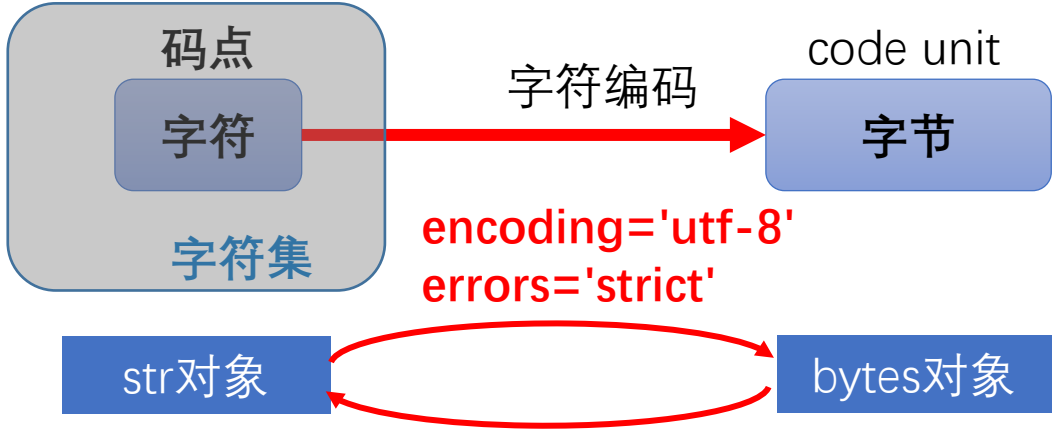
字符串str



unicode 字符串str与字节串bytes之间的转换

两类方法：当前对象的encode或decode方法；要转换为的对象的构造函数法

- encoding参数给出**编码方式**，缺省为utf-8，常用的编码还包括ascii, latin1, gbk等
- errors参数指出在编码/解码过程中**无法转换时怎么办？** 缺省 **'strict'**，即**抛出异常**。比如某些字符不属于相应字符集中的字符，或者某些字节不是合法的字节序列



含义	方法	例子
str→bytes	<code>str_obj.encode(encoding='utf-8', errors='strict')</code> <code>str_obj.encode(encoding='ascii', errors='replace')</code> <code>bytes(strobj, encoding='utf-8', errors='strict')</code>	<code>'C语言'.encode('gbk')</code> 采用GBK编码方式将字符串编码成字节串 <code>bytes('C语言', encoding='ascii', errors='replace')</code> 字符串转换成ASCII码，字符不能转换时用?代替
bytes→str	<code>bytesobj.decode(encoding='utf-8', errors='strict')</code> <code>str(bytesobj, encoding='utf-8', errors='strict')</code> 至少有两个参数，否则str(bytesobj)表示将bytesobj以友好方式转换为字符串	<code>'C语言'.encode('gbk').decode('gbk')</code> <code>str('C语言'.encode('gbk'), 'gbk')</code>

编码: unicode字符串→字节串

errors参数指出在编码过程中由于某些字符不属于相应字符集中的字符从而无法转换时怎么办

- strict(缺省):抛出异常UnicodeEncodeError
- ignore: 忽略并跳过无法编码的字符
- replace:无法编码时替换为缺省替换字符, 一般为?
- backslashreplace:无法编码时替换为通过\进行转义的转义序列
- xmlcharrefreplace: 无法编码时替换为XML表示方式
- namereplace: 替换为用名字描述的Unicode字符特性\N{...}

```
def test_unicode_encode():  
    s1 = 'I love Python语言!'  
    try:  
        print(s1.encode('ascii', errors='strict'))  
    except Exception as e:  
        print(e)  
    for error in ('ignore', 'replace',  
'backslashreplace', 'xmlcharrefreplace',  
'namereplace'):  
        print("encoding with errors '%s'" % error)  
        print(s1.encode('ascii', errors=error))
```

str_encoding.py

```
<class 'UnicodeEncodeError'> 'ascii' codec  
can't encode characters in position 13-14:  
ordinal not in range(128)
```

```
encoding with errors 'ignore'  
b'I love Python!'
```

```
encoding with errors 'replace'  
b'I love Python??!'
```

```
encoding with errors 'backslashreplace'  
b'I love Python\\u8bed\\u8a00!'
```

```
encoding with errors 'xmlcharrefreplace'  
b'I love Python&#35821;&#35328;!'
```

```
encoding with errors 'namereplace'  
b'I love Python\\N{CJK UNIFIED IDEOGRAPH-  
8BED}\\N{CJK UNIFIED IDEOGRAPH-8A00}!'
```

编码：字节串→unicode字符串

str对象

encoding='utf-8'
errors='strict'

bytes对象

bytes_obj.decode(encoding='utf-8', errors='strict') -> str

str(bytes_or_buffer[, encoding='utf-8'[, errors='strict']]) -> str

str(obj) -> str 将对象转换为面向用户的字符串

- 解码时也有可能出现无法解码的情况， errors可取值'strict','ignore','replace', 'backslashreplace'等
- errors取值'strict'， 解码出错时抛出异常 `UnicodeDecodeError`

```
>>> s1 = 'Python程序设计'
>>> bs1 = s1.encode() # 转换为字节串bs1, utf-8编码方式
>>> print(bs1)
b'Python\xe7\xa8\x8b\xe5\xba\x8f\xe8\xae\xbe\xe8\xae\xa1'
>>> s2 = str(bs1) # 将字节串转换为友好的字符串(知道其是一个bytes对象)
>>> print(s2)
b'Python\xe7\xa8\x8b\xe5\xba\x8f\xe8\xae\xbe\xe8\xae\xa1'
>>> s3 = str(bs1, 'utf-8') #或者 bs1.decode()
>>> print(s3)
Python程序设计
>>> s1.encode('gbk').decode('utf-8')
Traceback (most recent call last):...
UnicodeDecodeError: 'utf-8' codec can't decode byte 0xb3 in position 6: invalid start byte
```